
ML TEORÍA I

Modelos y generalización

Fernanda Sobrino

2026

¿Qué es aprender de los datos?

¿Qué es un modelo?

$$y = f(x)$$

Diagram illustrating the components of the equation $y = f(x)$:

- y : output
- $=$: "Learned" function
- $f(x)$: Features/variables/inputs/predictors/independent variables

- ▶ Un modelo es una función: le das características de un caso y te devuelve una predicción
- ▶ No es una teoría, no es una explicación, no es una ley: es una regla de asociación ajustada a unos datos concretos
- ▶ La pregunta que contesta siempre tiene esta forma: dado lo que sé de este caso, ¿qué esperaría de la variable que no observo?

Aprender de los datos, no escribir reglas

$f(\text{🍏}) = \text{"apple"}$

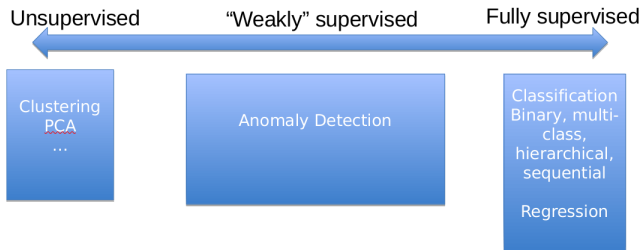
$f(\text{🐕}) = \text{"dog"}$

- ▶ La programación clásica: tú escribes la regla, la computadora la aplica
- ▶ El aprendizaje de máquina: tú le das ejemplos con la respuesta, y la computadora escribe la regla
- ▶ Por eso funciona donde la regla es imposible de escribir a mano (¿esto es una manzana? ¿este trámite es fraudulento?)
- ▶ Y por eso falla de formas raras: la regla que aprendió puede no ser la que tú creías

Los tres ingredientes

1. **Features** (X): lo que sabes de cada caso — edad, ingreso, historial de inspecciones, municipio
 2. **Etiqueta** (y): lo que quieres predecir — sobrevivió / no sobrevivió, monto, riesgo alto / bajo
 3. **Algoritmo**: la receta que busca la f tal que $f(X) \approx y$
- ▶ Gran parte del éxito de un modelo depende de haber construido las features correctas, no de haber elegido el algoritmo de moda
 - ▶ Sin etiqueta no hay aprendizaje supervisado. “Quiero predecir corrupción” no es una etiqueta; “esta obra fue sancionada por la ASF en los 2 años siguientes” sí lo es

Supervisado y no supervisado



- ▶ **Supervisado:** tenemos historia etiquetada, sabemos la respuesta correcta de casos pasados y podemos supervisar la predicción
- ▶ **No supervisado:** no hay respuesta correcta; buscamos estructura, patrones, grupos
- ▶ La diferencia práctica: en supervisado se puede medir el error; en no supervisado, no del todo

Regresión o clasificación

Si la y es **numérica** el problema es de regresión; si es **categorica**, de clasificación.

Pregunta	Tipo
¿Este trámite tiene documentación falsa?	Clasificación
¿Cuántos alumnos se inscribirán el próximo ciclo?	Regresión
¿Este embarazo es de alto riesgo?	Clasificación
¿Cuánto tardará este expediente en resolverse?	Regresión
¿Esta imagen satelital muestra tala ilegal?	Clasificación

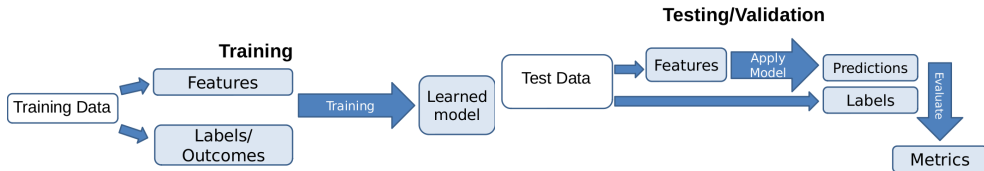
- La misma pregunta puede plantearse de las dos formas — y la decisión la tomas tú, no el algoritmo

Clasificación binaria y la etiqueta positiva

- ▶ El caso más común en política pública: dos categorías, y una nos interesa más
- ▶ La etiqueta positiva (el “1”) es el evento que queremos encontrar: el fraude, la deserción, la violación a la norma
- ▶ Casi siempre es la rara: 2 %, 0.5 %, 0.1 % de los casos. Eso va a dominar todo lo que hagamos después
- ▶ Muchos modelos no devuelven un 1 o un 0, sino un número entre 0 y 1: un puntaje. Para actuar hay que decidir un corte

Generalizar

El objetivo no es explicar el pasado



- ▶ Un modelo que reproduce perfectamente los datos que ya tienes no sirve de nada: esos casos ya los conoces
- ▶ Lo que importa es el desempeño en datos que el modelo nunca vio — porque así se va a usar
- ▶ Podríamos hacer una prueba de campo y esperar seis meses... o podríamos simularla hoy

Train y test

- ▶ Partimos los datos: típicamente 80 % entrenamiento, 20 % prueba
- ▶ El modelo se entrena solo con el 80 %. El 20 % restante lo guardamos y no lo tocamos
- ▶ Al final medimos el desempeño en ese 20 % — que para el modelo son datos del futuro
- ▶ El test es una simulación honesta de la puesta en producción

Cuando los datos tienen tiempo

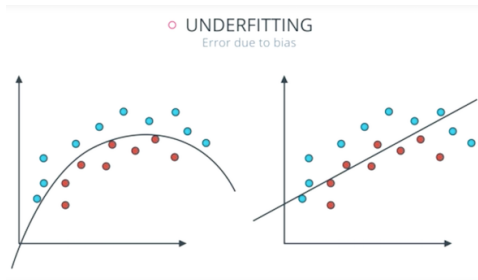
- ▶ Inspecciones, trámites, delitos, deserción escolar: casi todo en política pública tiene fecha
- ▶ Partir al azar significa entrenar con enero de 2024 y evaluar con marzo de 2023: el modelo ve el futuro
- ▶ Con datos temporales el split es por fecha de corte: entreno con el pasado, evalúo con el futuro
- ▶ Esto imita cómo se va a usar el modelo en la realidad — y siempre da métricas más bajas (y más honestas)

Sobreajuste



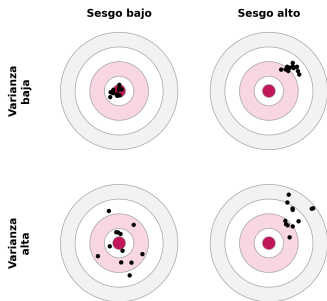
- ▶ El modelo memoriza el ruido de los datos de entrenamiento: cada punto, cada excepción
- ▶ Excelente en train, malo en test. Aprendió los casos, no el patrón
- ▶ Ocurre cuando el modelo es demasiado flexible para la cantidad de datos que tienes

Subajuste



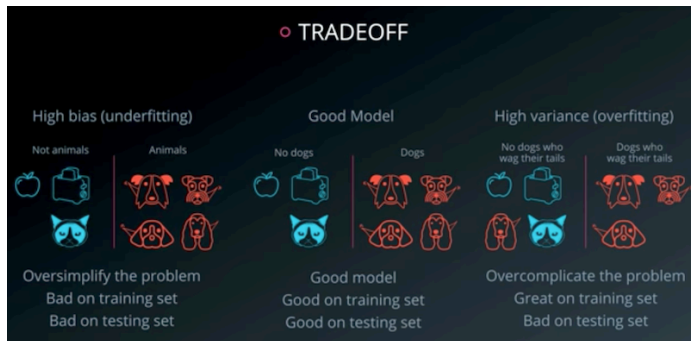
- ▶ El modelo es demasiado simple para el patrón que hay: ni siquiera aprende lo que sí está
- ▶ Malo en train y malo en test — que es, al menos, un síntoma fácil de ver
- ▶ Con muy pocos datos, un modelo simple es la decisión correcta, no un error

Dos nombres para esos dos errores



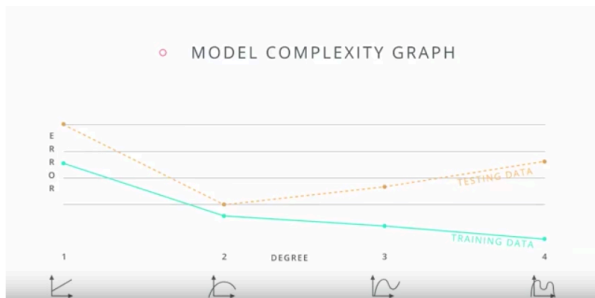
- Imagina entrenar el mismo modelo muchas veces, cada vez con una muestra distinta de datos: cada punto es uno de esos modelos; el centro es el patrón real
- **Sesgo:** qué tan lejos del centro caes en promedio. Es el error del modelo demasiado simple: ni con datos infinitos le atinaría (subajuste)
- **Varianza:** qué tanto cambia tu tiro de muestra a muestra. Es el error del modelo que memoriza: cada muestra le cuenta un ruido distinto (sobreajuste)

El balance



- ▶ Demasiado simple: sesgo alto. Demasiado flexible: varianza alta
- ▶ No hay un modelo “correcto” en abstracto: hay un punto de complejidad adecuado para estos datos

Cómo se ve en una gráfica



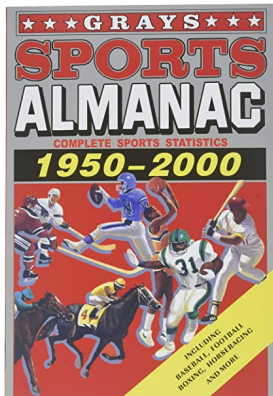
- El error en train baja siempre que subes complejidad. El error en test baja... y luego sube
- Ese punto donde el error de test empieza a subir es el límite
- Por eso hace falta el test: sin él, la curva verde te haría creer que el modelo mejora sin parar

Cómo se controla el sobreajuste

- ▶ **Limitar la complejidad:** profundidad máxima de un árbol, número de vecinos en KNN, regularización
- ▶ **Más datos:** casi siempre lo mejor, casi nunca lo posible
- ▶ **Menos features:** cientos de variables con cientos de observaciones es una receta de memorización.
- ▶ **Features:** cuidado casi siempre faltan features, esto no es un problema de causalidad no hay que poner 3 cosas que creo explican el fenómeno.
- ▶ **Validación cruzada:** partir el train en varios pedazos para elegir hiperparámetros sin tocar el test
- ▶ El síntoma que siempre debe encender la alarma: un desempeño **demasiado bueno**

Fuga de datos

Fuga de datos (data leakage)



Fuga: información que el modelo no tendría disponible al momento de predecir se cuela al entrenamiento.

Los tres casos clásicos

1. **Escalar (o imputar, o codificar) antes del split** — el test le pasa su media al train
2. **Variables que conocen el futuro** — el almanaque deportivo de 1950 a 2000
3. **Duplicados entre train y test** — el modelo ya vio la respuesta del examen

Las tres se ven igual desde afuera: un modelo con resultados demasiado buenos.

Caso 1: transformar antes de partir

- ▶ Muchas transformaciones aprenden números de los datos: escalar usa la media y la desviación, imputar usa la mediana, codificar usa las categorías que existen
- ▶ Si calculas esos números con todos los datos y después partes, el train ya sabe algo del test: su media, su rango, sus faltantes. El examen se filtró al estudio
- ▶ El orden correcto: primero partir; después aprender la transformación solo con train; y aplicarla, ya fija, al test
- ▶ Aplica igual a imputar faltantes, codificar categorías, seleccionar variables y hasta a quitar outliers
- ▶ Es la fuga más frecuente y la más fácil de detectar: cualquier cálculo hecho sobre los datos completos antes del split

Caso 2: variables que conocen el futuro

- ▶ Predecir si un paciente será hospitalizado usando `dias_de_hospitalizacion`
- ▶ Predecir deserción escolar usando `motivo_de_baja`
- ▶ Predecir si una empresa será sancionada usando `monto_de_la_multa`
- ▶ Predecir demanda del programa usando el padrón actualizado después del cierre

¿Este dato ya existía, y con este valor, el día en que hay que tomar la decisión?

- ▶ Si la respuesta es no, la variable se va. Aunque sea la más predictiva.

Caso 3: duplicados entre train y test

- ▶ El mismo trámite capturado dos veces, el mismo establecimiento con dos IDs, el mismo hogar en dos rondas de encuesta
- ▶ Si una copia cae en train y la otra en test, el modelo no predice: recuerda
- ▶ También pasa con panel: si una entidad tiene 12 renglones mensuales, el split al azar la parte a la mitad
- ▶ La solución: quitar duplicados antes, y con datos panel partir por entidad, no por renglón

Cómo detectarla

- ▶ Revisa correlaciones absurdamente altas entre una feature y la etiqueta
- ▶ Revisa fechas: ¿alguna variable tiene fecha posterior a la etiqueta?
- ▶ Y sobre todo: desconfía de los buenos resultados. Un AUC de 0.99 no es un modelo genial probablemente es una fuga

El orden de las operaciones (el resumen de todo)

Orden	Paso	Con qué datos
1	Partir en train y test	todos, una sola vez
2	Aprender las transformaciones (escalar, imputar, codificar)	solo train
3	Aplicar esas transformaciones	train y test
4	Entrenar el modelo	solo train
5	Elegir hiperparámetros y umbral	validación (nunca test)
6	Evaluar	solo test, una vez

► Si un paso “aprende” algo de los datos, va después del split